🚀 Collections Framework Masterclass

Chapter 14 — Map Interface — Complete Deep Dive

> **Interview Importance:** ⭐⭐⭐⭐⭐

We are now entering the **most important part of the Collections Framework for interviews**.

`Map` is fundamentally different from `List`, `Set`, `Queue`, and `Deque`.

The key idea is:

> **A Map stores data as key-value pairs. Keys are unique; values don't have to be.**

---

1. What is Map?

Package:

```
java.util.Map
```

A Map represents:

```
Key → Value
```

Example:

```
101 → "Thimmaraju"
102 → "Rahul"
103 → "Priya"
```

Here:

```
Keys   → 101, 102, 103
Values → Thimmaraju, Rahul, Priya
```

---

2. Map Is NOT a Collection

This is a common interview question.

The hierarchy is:

```
Iterable
   ↓
Collection
   ├── List
   ├── Set
   └── Queue

Map
   ├── HashMap
   ├── LinkedHashMap
   ├── TreeMap
   └── ...
```

`Map` does **not** extend `Collection`.

Why?

Because a Collection represents individual elements:

```
A
B
C
```

A Map represents associations:

```
A → 100
B → 200
C → 300
```

However, a Map provides Collection views such as:

```
map.keySet();
map.values();
map.entrySet();
```

---

3. Basic Map Example

```
Map<Integer, String> employees =
    new HashMap<>();

employees.put(101, "John");
employees.put(102, "David");
employees.put(103, "Sarah");
```

Conceptually:

```
101 → John
102 → David
103 → Sarah
```

---

4. Keys Must Be Unique

```
Map<Integer, String> map =
    new HashMap<>();

map.put(101, "John");
map.put(102, "David");
map.put(101, "Mike");
```

Final result:

```
101 → Mike
102 → David
```

The second `put(101, ...)` **replaces** the previous value.

It does not create another entry.

---

5. Values Can Be Duplicated

```
map.put(101, "John");
map.put(102, "John");
map.put(103, "John");
```

Valid:

```
101 → John
102 → John
103 → John
```

So remember:

```
Key   → Unique
Value → Can repeat
```

---

6. Core Map Methods

The most important methods:

```
put()
get()
remove()
containsKey()
containsValue()
size()
isEmpty()
clear()
```

And views:

```
keySet()
values()
entrySet()
```

---

7. `put()`

Adds or updates a mapping.

```
map.put("A", 100);
```

If `"A"` doesn't exist:

```
A → 100
```

If it already exists:

```
map.put("A", 200);
```

Then:

```
A → 200
```

---

8. Return Value of `put()`

Important interview detail.

```
Map<String, Integer> map =
    new HashMap<>();

System.out.println(
    map.put("A", 100)
);

System.out.println(
    map.put("A", 200)
);
```

Output:

```
null
100
```

Why?

First insertion:

```
No previous value → null
```

Second insertion:

```
Previous value → 100
```

`put()` returns the **previous value associated with the key**, or `null` if there was none.

---

9. `get()`

Retrieves a value:

```
Integer value =
    map.get("A");
```

If present:

```
A → 200
```

returns:

```
200
```

If the key isn't present:

```
map.get("X");
```

returns:

```
null
```

---

10. The `get()` + null Problem

Consider:

```
map.put("A", null);
```

Then:

```
map.get("A");
```

returns:

```
null
```

But:

```
map.get("X");
```

also returns:

```
null
```

So you cannot always distinguish:

```
Key exists with null value
```

from:

```
Key doesn't exist
```

using `get()` alone.

Use:

```
map.containsKey("A");
```

---

11. `containsKey()`

Checks whether a key exists.

```
if (map.containsKey("A")) {
    ...
}
```

This is generally the correct operation when your question is:

> "Does this key exist?"

---

12. `containsValue()`

Checks whether a value exists:

```
map.containsValue("John");
```

Unlike key lookup, value searching generally requires examining entries.

Typical complexity:

```
O(n)
```

Therefore:

```
containsKey()
→ designed around key lookup

containsValue()
→ typically scans values
```

---

13. `remove()`

Remove by key:

```
map.remove(101);
```

The entire mapping disappears:

```
101 → John
```

is removed.

`remove(key)` returns the previous value, or `null` if there was no mapping.

---

14. `remove(key, value)`

There is also a conditional version:

```
map.remove(101, "John");
```

It removes the mapping **only if both key and value match**.

This is useful for conditional updates and concurrent-style logic.

---

15. `replace()`

Replace an existing value:

```
map.replace(101, "Mike");
```

Unlike `put()`, `replace()` only operates when the key already exists.

There is also:

```
map.replace(
    101,
```

```
    "John",
    "Mike"
);
```

This replaces only when the current value matches `"John"`.

---

16. `putIfAbsent()`

Very important modern Map method.

```
map.putIfAbsent(
    101,
    "John"
);
```

It inserts only if the key doesn't already have a mapping.

Conceptually:

```
Key exists?
   ↓
 YES → do nothing
 NO  → insert
```

This is particularly useful when you don't want to overwrite an existing value.

---

17. `computeIfAbsent()`

One of the most useful Map methods.

Suppose:

```
Map<String, List<Integer>> map =
    new HashMap<>();
```

You want a list for a key, creating it only when necessary.

Instead of verbose code:

```
if (!map.containsKey("Java")) {
    map.put("Java", new ArrayList<>());
}

map.get("Java").add(10);
```

Use:

```
map.computeIfAbsent(
    "Java",
    k -> new ArrayList<>()
).add(10);
```

This pattern is extremely common in real Java code.

---

18. `computeIfPresent()`

Runs a computation only when the key is already mapped.

```
map.computeIfPresent(
    "A",
    (key, value) -> value + 10
);
```

If:

```
A → 100
```

becomes:

```
A → 110
```

---

19. `compute()`

Runs the remapping function regardless of whether the key currently exists.

```
map.compute(
    "A",
    (key, value) ->
        value == null ? 1 : value + 1
);
```

Useful for counters and state updates.

---

20. `merge()`

Very useful for counting.

Suppose:

```
Map<String, Integer> counts =
    new HashMap<>();
```

Instead of:

```
if (counts.containsKey(word)) {
    counts.put(
        word,
        counts.get(word) + 1
    );
} else {
    counts.put(word, 1);
}
```

Use:

```
counts.merge(
    word,
    1,
    Integer::sum
);
```

For:

```
Java Java Python
```

result:

```
Java   → 2
Python → 1
```

---

21. `keySet()`

Returns a Set view of the keys.

```
Set<Integer> keys =
    map.keySet();
```

Example:

```
Map:

101 → John
102 → David
103 → Sarah
```

`keySet()`:

```
101
102
103
```

Because keys are unique, a Set is appropriate.

---

22. `values()`

Returns a Collection view of values.

```
Collection<String> values =
    map.values();
```

Example:

```
John
David
Sarah
```

Why `Collection` instead of `Set`?

Because values can be duplicated.

---

23. `entrySet()`

Returns Set of key-value entries.

```
Set<Map.Entry<Integer, String>>
    entries = map.entrySet();
```

Each entry contains:

```
key
value
```

Example:

```
101 → John
102 → David
```

---

24. Best Way to Iterate a Map

Prefer:

```
for (Map.Entry<Integer, String> entry
        : map.entrySet()) {
```

```
    System.out.println(
        entry.getKey()
        + " = "
        + entry.getValue()
    );
}
```

Why?

Because you get both:

```
key + value
```

directly.

---

25. Why Not `keySet()` + `get()`?

You could do:

```
for (Integer key : map.keySet()) {
    System.out.println(
        key + " = " + map.get(key)
    );
}
```

But this performs a separate lookup for every key.

With:

```
entrySet()
```

you directly access the mapping.

Therefore:

> When you need both keys and values, prefer `entrySet()`.

---

26. Map Views Are Backed by the Map

This is an important concept.

```
Set<Integer> keys =
    map.keySet();
```

This isn't necessarily an independent copy.

It is a **view**.

Therefore modifications through supported view operations can affect the original Map.

Conceptually:



These are different views of the same underlying mappings.

---

27. Map Implementations

The major implementations you need to know:

```
HashMap
LinkedHashMap
TreeMap
Hashtable
ConcurrentHashMap
WeakHashMap
IdentityHashMap
EnumMap
```

Their selection depends on requirements.

---

28. HashMap

The most commonly used Map.

Characteristics:

```
Key-value storage
+
Fast average lookup
+
No ordering guarantee
```

Typical:

```
put()    → O(1) average
get()    → O(1) average
remove() → O(1) average
```

Internally:

```
HashMap
   ↓
Hash table
   ↓
Buckets
```

We'll dedicate a separate chapter to HashMap internals.

---

29. LinkedHashMap

Adds predictable iteration order.

Conceptually:

```
HashMap
+
Linked ordering structure
```

By default, it maintains insertion order.

Example:

```
Map<Integer, String> map =
    new LinkedHashMap<>();

map.put(3, "C");
map.put(1, "A");
map.put(2, "B");
```

Iteration:

```
3 → C
1 → A
2 → B
```

---

30. TreeMap

Provides sorted keys.

Internally:

```
TreeMap
   ↓
Red-Black Tree
```

Example:

```
Map<Integer, String> map =
    new TreeMap<>();

map.put(30, "C");
map.put(10, "A");
map.put(20, "B");
```

Iteration:

```
10 → A
20 → B
30 → C
```

Typical operations:

```
O(log n)
```

---

31. Hashtable

Legacy synchronized Map implementation.

```
Hashtable<Integer, String> map =
    new Hashtable<>();
```

Characteristics:

- Thread-safe through synchronization
- Does not allow null keys
- Does not allow null values
- Legacy API

For new concurrent code, prefer appropriate classes from `java.util.concurrent`, especially `ConcurrentHashMap` for general concurrent hash-map use.

---

32. ConcurrentHashMap

Designed for concurrent access.

```
Map<Integer, String> map =
    new ConcurrentHashMap<>();
```

Important characteristics:

- Thread-safe
- High concurrency
- Does not allow null keys
- Does not allow null values

- Better suited to modern concurrent applications than Hashtable

We'll study its internals separately.

---

33. WeakHashMap

Uses **weak references for keys**.

This allows entries to become eligible for garbage collection when their keys are no longer strongly referenced elsewhere.

Conceptually:

```
Normal HashMap:

Map → Key
     ↓
  Key stays reachable

WeakHashMap:

Map → weak reference → Key
```

Useful for certain cache/metadata scenarios.

It is not a general-purpose replacement for a normal cache.

---

34. IdentityHashMap

This is a particularly interesting Map.

Normal HashMap compares keys using:

```
equals()
```

IdentityHashMap uses:

```
==
```

Example:

```
String a = new String("Java");
String b = new String("Java");
```

Then:

```
a.equals(b)
```

is:

```
true
```

but:

```
a == b
```

is:

```
false
```

HashMap considers them equal keys.

IdentityHashMap treats them as different keys.

---

35. EnumMap

Specialized Map for enum keys.

```
enum Day {
    MONDAY, TUESDAY, WEDNESDAY
}
```

Then:

```
EnumMap<Day, String> map =
    new EnumMap<>(Day.class);
```

Advantages:

- Very efficient
- Compact
- Designed specifically for enum keys
- Maintains enum declaration order when iterated

If your keys are enums, `EnumMap` is often an excellent choice.

---

36. Map Selection Guide

This table is worth memorizing:

Requirement	Choose
General-purpose fast lookup	`HashMap`
Fast lookup + insertion order	`LinkedHashMap`
Sorted keys	`TreeMap`
Concurrent hash map	`ConcurrentHashMap`
Legacy synchronized map	`Hashtable`
Keys should be weakly referenced	`WeakHashMap`
Identity (`==`) key comparison	`IdentityHashMap`
Enum keys	`EnumMap`

---

37. HashMap vs LinkedHashMap vs TreeMap

Feature	HashMap	LinkedHashMap	TreeMap
Ordering	None guaranteed	Insertion/access order	Sorted keys
Typical get	O(1) avg	O(1) avg	O(log n)
Typical put	O(1) avg	O(1) avg	O(log n)
Structure	Hash table	Hash table + links	Red-Black Tree

Feature	HashMap	LinkedHashMap	TreeMap
Null key	One allowed	One allowed	Generally no
Duplicates keys	No	No	No

---

38. Map Does Not Mean Duplicate Keys

This:

```
map.put("A", 10);
map.put("A", 20);
```

doesn't produce:

```
A → 10
A → 20
```

It produces:

```
A → 20
```

If you need multiple values per key:

```
Map<String, List<Integer>>
```

is a common design.

Example:

```
Java → [10, 20, 30]
Python → [40, 50]
```

---

39. Map of Lists Pattern

```
Map<String, List<Integer>> map =
    new HashMap<>();

map.computeIfAbsent(
    "Java",
    k -> new ArrayList<>()
).add(10);

map.computeIfAbsent(
    "Java",
    k -> new ArrayList<>()
).add(20);
```

Result:

```
Java → [10, 20]
```

This pattern appears frequently in real projects and coding interviews.

---

40. `forEach()`

Modern Java provides:

```
map.forEach(
    (key, value) ->
        System.out.println(
            key + " = " + value
        )
);
```

Concise and readable.

---

41. `getOrDefault()`

Useful when a default value is required.

```
int count =
    map.getOrDefault(
        "Java",
        0
    );
```

If:

```
Java → 5
```

returns:

```
5
```

If absent:

```
0
```

This is useful for counters and lookup logic.

---

42. `replaceAll()`

Allows transformation of all values.

```
map.replaceAll(
    (key, value) -> value + 10
);
```

If:

```
A → 100
B → 200
```

becomes:

```
A → 110
B → 210
```

---

43. `clear()`

Removes all mappings:

```
map.clear();
```

Result:

```
{}
```

---

44. `size()`

Returns number of key-value mappings:

```
map.size();
```

If:

```
A → 10
B → 20
C → 30
```

then:

```
size() → 3
```

Not 6.

A mapping consists of one key and one value.

---

45. `isEmpty()`

Checks:

```
map.isEmpty();
```

Returns:

```
true
```

if there are no mappings.

---

46. Map and `equals()` / `hashCode()`

For hash-based Maps such as HashMap, key lookup depends on hashing and equality.

Conceptually:

```
hashCode()
    ↓
bucket
    ↓
equals()
    ↓
matching key
```

This is why mutable keys can be dangerous.

---

47. Mutable Key Problem

Suppose:

```
Map<Employee, String> map =
    new HashMap<>();
```

If `Employee.hashCode()` depends on:

```
employee.id
```

and you change the ID after insertion:

```
employee.id = 999;
```

the key's hash can change.

Now the Map may be unable to find the entry using the mutated key.

Therefore:

> **Keys used in hash-based Maps should generally be immutable with respect to fields involved in `equals()` and `hashCode()`.**

This becomes very important in the HashMap chapter.

---

48. Map and Null

Null behavior differs by implementation.

Map	Null Key	Null Value
HashMap	✅ One	✅
LinkedHashMap	✅ One	✅
TreeMap	Generally ❌	✅
Hashtable	❌	❌
ConcurrentHashMap	❌	❌
WeakHashMap	✅	✅
IdentityHashMap	✅	✅
EnumMap	❌	✅

The exact behavior should always be considered implementation-specific.

---

49. ⭐ Interview Scenario

Requirement

You need:

```
Key → Employee
```

and want the fastest average lookup.

No ordering requirement.

Answer:

```
Map<Integer, Employee> employees =
    new HashMap<>();
```

---

50. ⭐ Interview Scenario

Requirement

You need:

```
Key → Value
```

with insertion order preserved.

Answer:

```
Map<Integer, Employee> employees =
    new LinkedHashMap<>();
```

---

51. ⭐ Interview Scenario

Requirement

You need keys automatically sorted.

Answer:

```
Map<Integer, Employee> employees =
    new TreeMap<>();
```

---

52. ⭐ Interview Scenario

Requirement

Multiple threads access and update a Map.

Need a general-purpose concurrent hash-based implementation.

Answer:

```
Map<Integer, Employee> employees =
    new ConcurrentHashMap<>();
```

Don't simply use:

```
HashMap
```

without external synchronization.

---

53. ⭐ Interview Scenario

Requirement

Keys are enum values.

Answer:

```
EnumMap<MyEnum, String>
```

rather than automatically defaulting to HashMap.

---

54. Frequently Asked Interview Questions

Q1. Is Map a Collection?

No.

`Map` is a separate hierarchy.

Q2. Can Map contain duplicate keys?

No.

A key maps to at most one value at a time.

Q3. Can Map contain duplicate values?

Yes.

Q4. What happens when you put the same key twice?

The new value replaces the previous value.

Q5. Difference between `keySet()` and `entrySet()`?

```
keySet()
→ keys only

entrySet()
→ key + value
```

Q6. Why does `values()` return Collection rather than Set?

Because duplicate values are allowed.

Q7. Why is `entrySet()` preferred when iterating keys and values?

It provides each key-value mapping directly without performing a separate lookup.

Q8. Why is HashMap generally O(1)?

Hashing allows the implementation to locate the appropriate bucket directly on average.

Q9. Why is TreeMap O(log n)?

It uses a balanced Red-Black Tree.

Q10. HashMap vs LinkedHashMap?

HashMap has no ordering guarantee; LinkedHashMap maintains predictable iteration order.

Q11. HashMap vs TreeMap?

HashMap uses hashing and provides O(1) average basic operations.

TreeMap uses a balanced tree and provides sorted keys with O(log n) operations.

Q12. HashMap vs ConcurrentHashMap?

HashMap is not thread-safe.

ConcurrentHashMap is designed for concurrent access.

Q13. Why can't ConcurrentHashMap store null?

Because null would create ambiguity between:

```
key absent
```

and:

```
key mapped to null
```

and its concurrent APIs rely on this distinction.

Q14. What is WeakHashMap?

A Map whose keys are weakly referenced, allowing entries to become eligible for garbage collection when keys are no longer strongly reachable elsewhere.

Q15. What is IdentityHashMap?

A Map that uses reference identity (`==`) rather than normal object equality (`equals()`) for key comparison.

---

55. ⭐ Tricky Coding Question

What is the output?

```
Map<String, Integer> map =
    new HashMap<>();

map.put("A", 10);
map.put("B", 20);
map.put("A", 30);

System.out.println(map.size());
System.out.println(map.get("A"));
```

Answer:

```
2
30
```

Because `"A"` is a duplicate key.

---

56. ⭐ Tricky Coding Question

What happens?

```
Map<String, Integer> map =
    new HashMap<>();

map.put("A", null);
```

```
System.out.println(
    map.get("A")
);

System.out.println(
    map.get("B")
);
```

Both print:

```
null
```

To distinguish them:

```
map.containsKey("A");
```

returns:

```
true
```

while:

```
map.containsKey("B");
```

returns:

```
false
```

---

57. ⭐ Tricky Coding Question

```
Map<Integer, String> map =
    new HashMap<>();

map.put(1, "A");
map.put(2, "A");
map.put(3, "B");
```

What is:

```
map.size();
```

Answer:

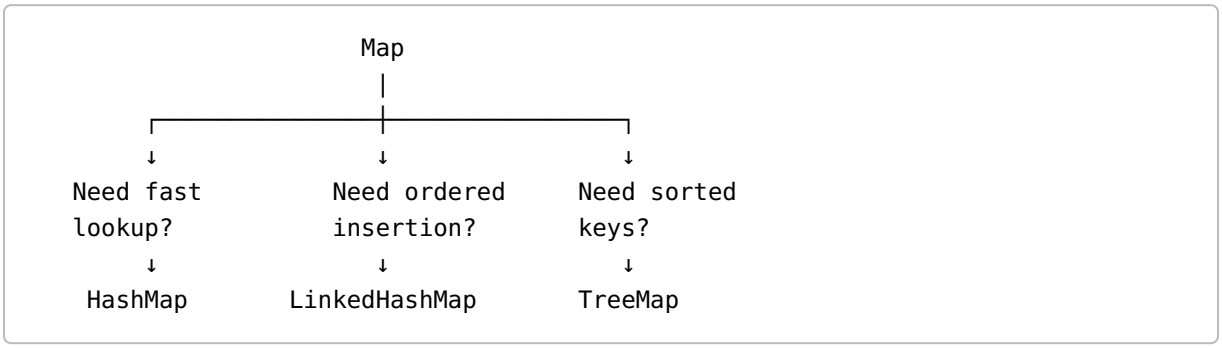
```
3
```

Duplicate **values** don't matter.

Only duplicate keys are replaced.

---

58. The Map Decision Framework



For special requirements:

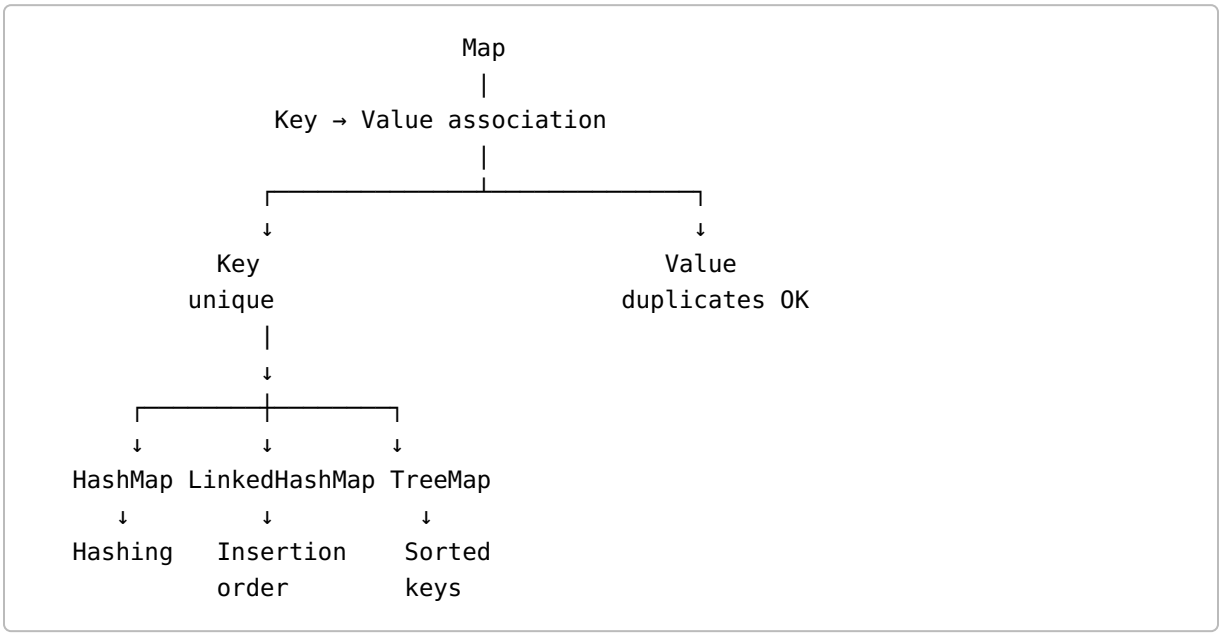
```
Concurrent access
       ↓
ConcurrentHashMap

Enum keys
       ↓
EnumMap

Weakly referenced keys
       ↓
WeakHashMap

Reference identity comparison
       ↓
IdentityHashMap
```

🧠 Final Mental Model



And the most important Map views:

```
keySet()
   ↓
Keys

values()
   ↓
Values

entrySet()
   ↓
Key + Value
```

🔥 Chapter Summary

Concept	Key Point
Map	Key-value association
Extends Collection?	❌

Concept	Key Point
Duplicate keys	❌
Duplicate values	✅
`put()`	Add/update mapping
`get()`	Retrieve value
`remove()`	Remove mapping
`containsKey()`	Check key
`containsValue()`	Check value
`keySet()`	Key view
`values()`	Value view
`entrySet()`	Entry view
`HashMap`	Fast average lookup
`LinkedHashMap`	Predictable order
`TreeMap`	Sorted keys
`ConcurrentHashMap`	Concurrent access
`WeakHashMap`	Weakly referenced keys
`IdentityHashMap`	`==` key comparison
`EnumMap`	Enum keys

🎯 Interview Readiness Checklist

You should be able to explain:

- [ ] Why Map is not a Collection
- [ ] Why keys must be unique
- [ ] Whether values can be duplicated
- [ ] What happens when the same key is inserted twice
- [ ] `put()` return value
- [ ] `get()` vs `containsKey()`
- [ ] `keySet()` vs `values()` vs `entrySet()`
- [ ] Why `entrySet()` is preferred for key-value iteration
- [ ] `putIfAbsent()`
- [ ] `computeIfAbsent()`

- [ ] `compute()`
- [ ] `merge()`
- [ ] HashMap vs LinkedHashMap vs TreeMap
- [ ] HashMap vs ConcurrentHashMap
- [ ] WeakHashMap
- [ ] IdentityHashMap
- [ ] EnumMap
- [ ] Why mutable Map keys are dangerous
- [ ] Null behavior across Map implementations

🚀 Next Chapter

Chapter 15 — HashMap (Complete Deep Dive)

This is one of the **highest-priority chapters in the entire Collections Framework**.

We'll go deep into:

- HashMap internal structure
- `Node<K,V>`
- Hashing
- Hash spreading
- Bucket calculation
- `hash & (n - 1)`
- Why capacity is a power of 2
- `put()` internals step-by-step
- `get()` internals step-by-step
- `equals()` + `hashCode()`
- Hash collisions
- Collision handling
- Linked list buckets
- Treeification
- `TREEIFY_THRESHOLD`
- `UNTREEIFY_THRESHOLD`
- Load factor
- Threshold
- Resizing
- Rehashing
- Why capacity is doubled
- Why HashMap is not thread-safe
- Concurrent modification
- Fail-fast iterators
- Java 7 vs Java 8+ HashMap differences
- Real interview coding scenarios

- **30+ HashMap interview questions**